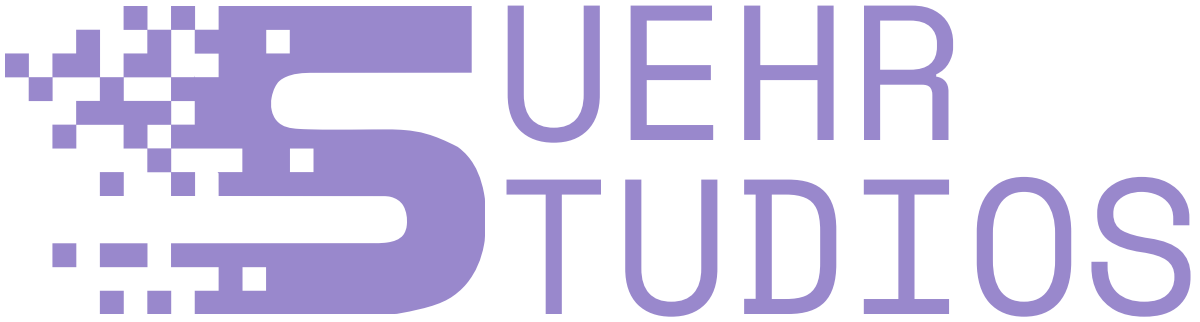


Learning the C3 Console

These notes exist so the owner of this project can explain it, defend it, and extend it without having built embedded systems before. They assume a strong software background (C++, domain-driven design, a game loop or two) and no background in electronics, cars, or Linux graphics below the window-system line. Nothing here replaces the build spec, which is the source of truth; these notes explain *why* the spec says what it says, in the order a newcomer needs it.

The two-minute version

A 1975 Corvette has a mechanical dash from the points-ignition era. We are adding a portrait touchscreen where the radio and heater controls used to be. The screen shows CarPlay in the middle and four gauges around it (oil pressure, coolant temperature, fuel level, system voltage), and it controls the blower, the A/C clutch, the vent/floor/defrost doors, and the headlamps.

Two computers do this, on purpose:

- **An ESP32-S3 microcontroller** reads the car. It taps the existing gauge senders without cutting anything, filters the readings, converts them to engineering units, decides when something is dangerous, and sounds a buzzer. It also drives the relays. It runs with the screen off, with the Pi crashed, with the UART cable unplugged.
- **A Raspberry Pi CM4** draws the screen. It receives numbers over a serial line, renders them with a hand-drawn NanoVG scene straight onto the display controller (no desktop, no X, no Wayland), hosts the CarPlay video stream, and sends *requests* to the microcontroller when you tap a button.

Nothing the Pi does can hurt the engine or leave you without headlights. Everything is a piggyback tap or a parallel relay, so the whole system comes out in an afternoon and the car

is stock again. That is the entire design, and every other decision follows from it.

Reading order

Section	What you learn
1. The Big Picture	The architecture, the two-computer split, and how it maps onto things you already know (bounded contexts, game loops, ports and adapters)
2. Electricity for Software Engineers	The dozen electronics ideas the rest of the project rests on: dividers, grounds, logic levels, relays, optocouplers, protection, power
3. Reading the Car Senders, Piggyback Tap & ADC	How a 1975 gauge circuit works, the piggyback tap, differential ADC measurement, filtering, and calibration
4. The Microcontroller	What a microcontroller is, why the ESP32-S3, how the firmware is structured, and how the warning path stays independent
5. The Wire Between Them UART, Framing & Failure	UART from first principles, the JSON-with-CRC framing, acknowledgements, failure behaviour, and record/replay
6. The Display Host Linux as an Appliance	Linux as an appliance: DRM/KMS/GBM/EGL, the event loop, the triple buffer, watchdog, read-only rootfs, CarPlay decode, audio
7. Controlling the Car HVAC, Lamps, Doors, Reverse	How the factory HVAC and lamp circuits work and how relays are added to them without taking anything away
8. Power and Grounding	The 12 V system that is not 12 V, the power tree, the master relay, star ground, graceful shutdown
9. Safety and Failure Thinking	The six invariants, the failure each one prevents, and the reversibility rule
10. Glossary	Every acronym and part name, one line each
11. Talking Points	How to explain the project at a whiteboard, and answers to the questions other engineers will ask

Read 01 and 02 first. After that, 03 through 08 stand alone; 09 and 11 are the ones to re-read before explaining the project to someone.

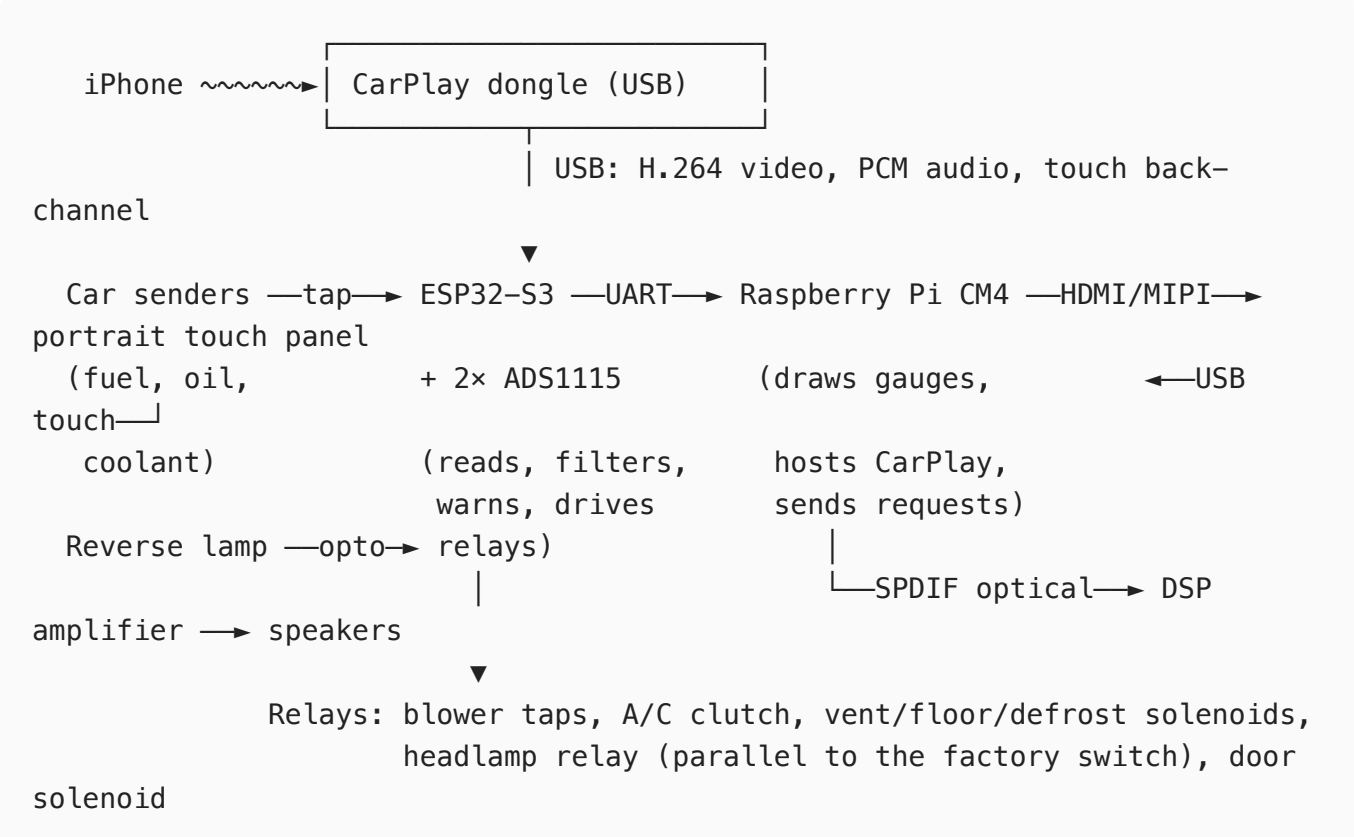
Conventions in these notes

- **Hypothesis** marks a number that comes from a datasheet, a forum, or the spec's initial guess, not from a measurement on this car. Sender resistance curves are the main example.
  Safety invariant 5 says these are never presented as calibrated, and the same rule applies to conversation: say "the nominal GM curve is..." not "the sender reads...".
- **Spec §n** points into `c3console-build-spec.md`. **Wiring notes** means `1975-corvette-wiring-notes.md`. **Bench** means `phase2-bench.md`.
- Code names are the ones in the tree today (`ConsoleScene`, `TelemetryIngestor`, `c3fw`). The developer guide (`developer-guide.md`) is the reference for the software itself; these notes only explain the hardware-facing reasoning behind it.

---

1. The Big Picture

What we are building, physically



Two boards, one wire between them, and every connection to the car is either a high-impedance tap (we listen without loading the circuit) or a relay contact wired in parallel with a switch that still works on its own.

Why two computers

A single Raspberry Pi has GPIO pins and could, in theory, do all of this. It should not, for four reasons that are worth being able to recite:

1. **Boot time and availability.** Linux boots in seconds; a microcontroller runs its first instruction a few milliseconds after power. The oil pressure warning has to be live the moment the key turns, and it has to keep working while the Pi is booting, rebooting after a watchdog reset, or lying on the bench with its eMMC corrupted.
2. **Determinism.** The Pi runs a preemptive general-purpose OS. A garbage collector is not the problem here, but a page-cache flush, a USB storm from the dongle, or a GPU stall is. The microcontroller runs one loop at 10 Hz with nothing else to do; its worst-case latency is knowable. The warning path and the relay outputs need that.
3. **Electrical exposure.** The senders live on the engine block at 12 V-system potentials with ignition noise on every wire. The board that touches them is a $15 part with $8 of ADCs and a handful of protection diodes. If a spike gets through, that board dies, not the $80 compute module with the display and the CarPlay session on it.
4. **Blast radius.** The Pi is where all the complexity is: a USB protocol reverse-engineered from other people's code, hardware video decode, a graphics stack, Wi-Fi. That code will crash sometimes. When it does, the blower keeps running at the speed you set, the buzzer still sounds on low oil pressure, and the headlights still answer the factory switch.

The split is expressed as a rule, not a preference: **the MCU owns everything safety-adjacent; the Pi is display, CarPlay and audio.** It is safety invariant 1 in `AGENTS.md`. The Pi never compares a reading against a threshold; it displays the fault list the MCU sent.

Mapping it onto things you already know

Bounded contexts. The MCU and the Pi are two bounded contexts with a published language between them: the newline-delimited JSON telemetry frame and the small command vocabulary of
spec §4. The wire protocol is the anti-corruption layer. Inside the Pi the `vehicle/` directory holds the shared value types (`HvacState` , `LampState` , `RelayState`) and the command/ack codec, exactly where a DDD design would put the shared kernel. The `Frame` struct
is a value object. `DockController` and `WellController` are the aggregates that own state transitions. Draw functions are pure views. There is no event bus, because two contexts do not need one.

Ports and adapters. Three abstract seams (`DisplayBackend` , `InputSource` , `SensorSource`) plus `CommandSink` , `AudioSink` and `Watchdog` are the ports. Each has a desktop adapter (SDL2 window, SDL mouse, `MockSource`) and a target adapter (KMS/GBM/EGL,
libinput, `UartSource`). The consequence the spec insists on: every feature is developed and

tested under `--desktop --source=mock` because that is where the hardware is not. Ninety percent of this project is built with the car untouched in the garage.

The game loop. If you have written a raylib game you already know the Pi's render loop: a fixed 60 Hz tick, gather input, update models, draw an immediate-mode scene, present. The differences are that the "physics" runs on another processor and arrives as messages; that a missed present is a fault condition (the watchdog notices); and that nothing in the loop may allocate, throw, or block. NanoVG is to this project what raylib's drawing API is to a game: paths, fills, text, no widget tree. The MCU firmware is the other half of a game loop: a fixed-step simulation at 10 Hz that owns the authoritative state.

Immediate mode versus retained mode. There is no scene graph and no UI framework. The scene is redrawn from the current snapshot every frame. This is deliberate: it means a stale snapshot can be *detected* and drawn as stale (grey gauges, "NO DATA") rather than a widget quietly keeping its last value. Safety invariant 4 is much easier to honour in immediate mode.

The simulation ladder

Spec §9 organises the whole project as tiers, each complete and testable, each one step closer to the car:

Tier	What runs	Car touched?
0	Browser mockup (`docs/c3-console-mockup*.html`)	No
1	Desktop C++ build with `MockSource` physics profiles	No
2	ESP32 + ADCs + potentiometers on a breadboard, real UART into the desktop app	No
3	Real GM senders on a hotplate and a hand pump; calibration tables	No
4	CSV replay of recorded frames; deterministic regression	No
5	Pi + panel + dongle on the bench at 12 V	No
6	Temporary in-car: taps, bracket, nothing cut	**Nothing irreversible**
7	Permanent install with a reproduction bezel	Yes

Today the software is at tiers 0, 1 and 4 with tier 2 firmware written and compiled but not yet on a board (parts list in the bench doc). Everything downstream is gated on hardware arriving, not on code.

Reversibility as an architectural constraint

The car is worth more than the console. Spec §13 turns that into engineering rules: every sender is read by a tap at the gauge terminal, factory gauges stay connected and functional, every relay is in parallel with a switch that stays wired, the original bezel is stored and a reproduction is cut. This is why there is no "replace the gauge cluster" option anywhere in the design, and why the correct answer to "why not just splice in" is always no.

One sentence per component

- **Senders**: 50-year-old variable resistors in the engine and tank. We read the voltage at the point where the factory gauge meets them.
- **ADS1115 ×2**: 16-bit I2C analog-to-digital converters; two because four *differential* channels need two chips.
- **ESP32-S3**: the microcontroller; samples, filters, converts, evaluates, warns, switches relays, and talks JSON over a serial line.
- **UART**: three wires (TX, RX, ground), 115200 baud, newline-delimited JSON with a CRC.
- **CM4**: the Linux compute module; draws with NanoVG directly on the display controller.
- **Panel**: 10–11", portrait, 1000-nit class, optically bonded, USB HID touch.
- **Dongle**: Carlinkit CPC200-CCPA, a USB device that speaks CarPlay to the phone and sends
  us H.264 video and PCM audio.
- **Relay board**: automotive 30 A relays with flyback diodes, driven by the MCU, wired in parallel with factory switches.
- **DSP amplifier**: takes optical SPDIF from the Pi so there is no analog ground path between the console and the amplifier.

---

2. Electricity for Software Engineers

Everything hardware-facing in this project rests on about a dozen ideas. None of them needs more than Ohm's law. Read this once with the diagrams in front of you and the rest of the notes become mechanical.

Ohm's law, and the only mental model you need

Voltage (V, volts) is a *difference* between two points, like altitude. Current (I, amperes) is flow. Resistance (R, ohms, Ω) is how much a path opposes flow. `V = I × R`.

Two consequences do all the work here:

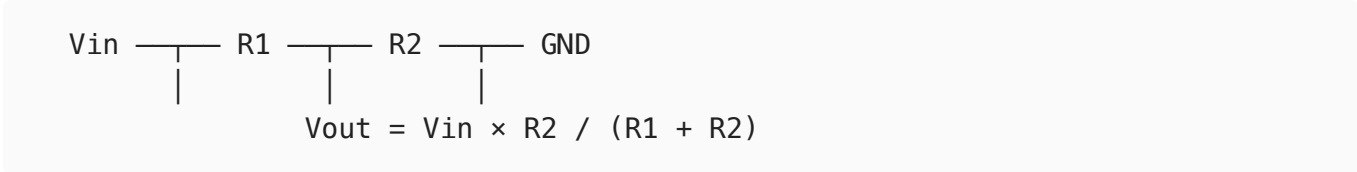
- **Current through a resistor drops voltage across it.** Put 1 kΩ in a path carrying 1 mA and the two ends differ by 1 V.

- **Voltage is always relative to something.** "12 V" means "12 V above whatever we call ground". If two things disagree about where ground is, they disagree about every voltage. That disagreement is the reason for half the design decisions in the sensor chain.

Power (W, watts) is `V × I`. A relay coil at 12 V drawing 150 mA dissipates 1.8 W. A blower motor at 12 V drawing 20 A is 240 W, which is why it gets a relay and not a transistor.

The voltage divider

Two resistors in series across a voltage produce, at the junction, a fraction of it:



This project uses it three ways:

1. **Measuring 12 V with a 3.3 V chip.** The system voltage channel uses 47 kΩ over 10 kΩ: `12 V × 10/57 = 2.1 V`, `16 V × 10/57 = 2.8 V`. Both inside the ADC's safe range.
2. **The factory gauge circuit itself is a divider.** The gauge is the top resistor, the sender is the bottom one, and the junction between them is what we tap. Doc 03 is about this.
3. **A potentiometer is an adjustable divider**, which is why four cheap 10 kΩ pots stand in for four senders on the bench.

Rule of thumb: the divider only works if whatever reads `Vout` draws negligible current compared to what flows through R1 and R2. An ADC input draws nanoamps, so it is fine. A factory
gauge draws tens of milliamps, so our tap does not disturb it either. This "high impedance listener" property is what makes the piggyback tap non-invasive.

The car's "12 V" is a range, not a number

A lead-acid battery rests at about 12.6 V. With the engine running the alternator holds the system at 13.8–14.4 V. During cranking the starter pulls it down to 9–10 V for a second. When a heavy load is switched off the alternator overshoots (a *load dump*, spec-grade automotive parts are rated for spikes to 100 V or more for tens of milliseconds). On top of all that sits ripple from the alternator and broadband noise from the ignition coil firing (1975 was GM's first HEI year; the spec's noise budget was written for a points car and is the more conservative assumption either way).

Consequences you will see in the design:

- The DC-DC converter for the Pi is specified "automotive-rated, 9–36 V in, load-dump rated". A generic USB car adapter is neither.
- A TVS (transient voltage suppressor) diode sits on the 12 V rail before the electronics. It conducts hard above its rated voltage and eats spikes.
- Every ADC input has a series resistor, a capacitor, and clamp diodes (below).
- The firmware median-filters every reading before doing anything else, because a single sample landing on an ignition pulse is normal, not exceptional.
- The `VOLT_LOW` / `VOLT_HIGH` fault thresholds (12.4 V, 15.2 V) and the "buzzer only while charging, ≥ 13.2 V" rule for oil pressure come straight from this range: 0 psi with the key on and the engine off is 12.6 V and normal; 0 psi at 14 V means the engine is running with no oil pressure.

Ground is not zero

Every current that flows out has to flow back. In a steel-bodied car the body is the return path and it is close enough to a single "ground". A C3 Corvette has a fibreglass body, so every ground is an actual wire back to the frame or the engine. Those wires have resistance and carry current, so two "grounds" ten feet apart can differ by 100–200 mV.

That matters because the fuel sender's whole range might be 0–90 Ω, which after the gauge's divider is a swing of perhaps a volt or two. A 200 mV ground error is a 10–20 % reading error that changes when you turn the headlights on. Two mitigations appear in the design:

- **Differential measurement** (doc 03): each ADC channel measures the voltage between its own plus and minus inputs, and the minus input goes to *that sender's* ground (engine block for oil and coolant, tank strap for fuel), not to the console's ground.
- **Star grounding** (doc 08): every board gets its own ground wire to one sanded-clean point on the steel birdcage, so no board's return current flows through another board's wire.

Logic levels, and why 12 V on a GPIO pin is fatal

Digital pins on both the ESP32 and the Pi are 3.3 V logic: 0 V is low, 3.3 V is high, and the absolute maximum they tolerate is about 3.6 V. Twelve volts, five volts, or a static discharge from a wool sweater on a pin will destroy it. Everything from the car therefore reaches a pin through something that limits it: a divider, an optocoupler, a relay contact.

Pull-ups and active-low. An input pin left unconnected reads noise. A *pull-up* resistor ties it weakly to 3.3 V so it reads high until something pulls it to ground. This is why switch inputs are usually "active-low": the switch shorts the pin to ground, and the firmware treats 0 as pressed. The reverse-gear sense input on the ESP32 (`pins::kReverseSense`) works
this way: pulled up, and the optocoupler pulls it low while reversing.

Active-high outputs. The relay board inputs are configured so that 3.3 V on the input means "coil on". The alternative (active-low, on = 0 V) would mean every relay clicks on when the MCU is unpowered or resetting, which is exactly wrong. The bench doc flags this as a jumper
to check.

Switching things: relay, transistor, optocoupler

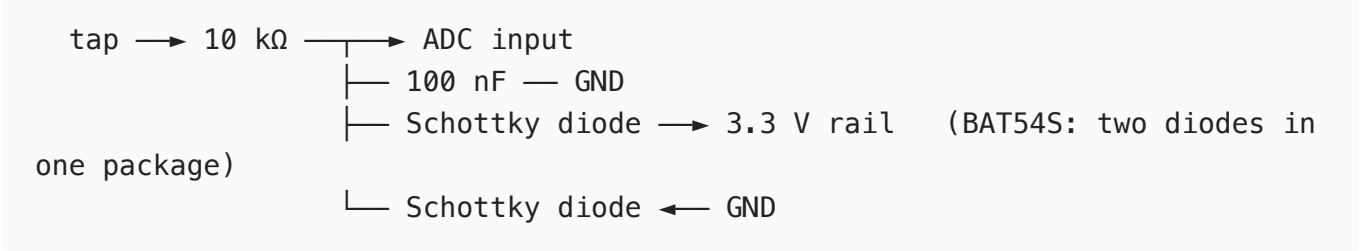
Relay. An electromagnet (the *coil*, pins 85 and 86 on an automotive relay) pulls a mechanical contact closed. The contact side (pin 30 common, 87 normally open, 87a normally closed) is completely separate from the coil: different voltage, different current, no electrical connection. That is why a 3.3 V logic board can switch a 20 A blower: the MCU drives a small transistor, the transistor energises the coil, the contacts carry the load. Relays are slow (~10 ms), audible, and wear out after ~100,000 operations, none of which matters for a blower switch. A **flyback diode** across the coil absorbs the voltage spike the coil generates when it is switched off; without it that spike kills the driver transistor.

Transistor as a switch. An NPN transistor with its emitter at ground turns on when the base is driven a fraction of a volt above ground through a resistor. The bench circuit for the warning buzzer is exactly this: GPIO → 1 kΩ → base, buzzer between 5 V and collector. The GPIO supplies a milliamp; the buzzer draws thirty. The relay module does the same job for the relay coils with an optocoupler in front.

Optocoupler. An LED and a light-sensitive transistor in one package, facing each other through plastic. Current in the LED turns the transistor on. Nothing electrical crosses the gap, so the two sides can sit at different grounds and a spike on one side cannot reach the other. The project uses optocouplers wherever a 12 V car signal has to be *sensed* by a 3.3 V input: the reverse lamp circuit into the MCU, the ignition feed into the Pi's shutdown GPIO, and the Pi's amplifier-remote output.

Protecting an analog input

Every ADC input in this design has the same four-part front end (spec §3.3):



- The **series resistor** limits current into the pin if something upstream goes to 12 V.

- The **capacitor** with the resistor forms a low-pass RC filter (corner ≈ 160 Hz) that takes the edge off ignition noise before the ADC sees it. `f = 1 / (2π R C)`.
- The **clamp diodes** conduct if the input tries to go above the 3.3 V rail or below ground, dumping the excess into the rail instead of the chip. Schottky diodes are used because they start conducting at ~0.3 V, before the chip's own weaker internal diodes would.

The bench doc's warning applies: the clamps save the chip from a mistake once, maybe. They are a seatbelt, not a design feature.

Power conversion and holding on through shutdown

A **linear regulator** turns 12 V into 5 V by burning the difference as heat; at 3 A that is 21 W of waste, unacceptable behind a dash. A **buck converter** switches the input on and off tens of thousands of times a second and smooths the result, at 90 %+ efficiency. Automotive buck modules add the wide input range and transient protection described above.

A **supercapacitor module** on the Pi's 5 V rail stores a few joules. When ignition drops, the Pi has 2–3 s of power to finish the frame, sync the data partition, and halt cleanly instead of being cut off mid-write. This matters less than it used to because the root filesystem is read-only (doc 06), but the settings and any in-car calibration live on a writable partition.

Fuses and wire

A fuse protects the **wire**, not the device. Every wire from the battery is fused within 18 inches of the battery at a current the wire can carry safely. Spec §8 uses a 60 A main fuse into 8 AWG wire, then a fuse block with 10 A, 10 A, 20 A and 40 A branches. Wire gauge (AWG, smaller number is thicker) is chosen from current and length: 8 AWG for the amplifier and the main feed, 12 AWG for the Pi and DC-DC ground, 16 AWG for the MCU board. Ground wires are sized
like the supply they return.

Digital buses in one paragraph each

- **I2C**: two wires (SDA data, SCL clock), shared by several chips each with a 7-bit address. The controller (ESP32) addresses a chip, then reads or writes registers. Slow (100–400 kHz),
  short-range (a board), fine for two ADCs. Both ADS1115s share the bus at 0x48 and 0x49, selected by strapping the ADDR pin low or high.
- **UART**: two wires (TX, RX) plus ground, no clock; both ends agree on a speed. Doc 05.
- **SPI**: faster than I2C, four wires, one chip-select per device. Not used here.
- **CAN**: the differential two-wire bus modern cars use between modules; robust and arbitrated, but needs transceivers on both ends and the 1975 car has none of it. Doc 11

answers "why not CAN".
- **USB**: the Pi's world, not the MCU's. The dongle, the touch digitiser, the camera and the microphone are all USB devices on the CM4.

---

3. Reading the Car: Senders, Piggyback Tap & ADC

This is the part of the project with the most physics and the least code. Get this right and the gauges are true; get it wrong and they are decorative.

How a 1975 gauge circuit works

Every analog gauge in the cluster is one half of a voltage divider. The gauge sits between the ignition-switched 12 V feed and a single wire that runs to a **sender** somewhere on the engine or in the tank. The sender is a variable resistor whose other end is grounded through the metal it is screwed into. Current flows feed → gauge → wire → sender → block → ground, and
the gauge needle position follows that current.

```
  IGN +12 V ──► [ factory gauge ] ──●── sender wire ──► [ sender R ] ──►
block/tank ──► ground
                                    │
                                    └── this node is what we tap
```

The three senders on this car (wiring notes, checked against the factory diagram):

Channel	Sender	Where	Wire colour	Nominal behaviour (hypothesis)
Fuel	Float on an arm moving a rheostat	In the tank, grounded through the tank strap	Tan	0 Ω empty → ~90 Ω full
Oil pressure	Diaphragm moving a wiper	Block, 1/8" NPT; **electric from 1975**, earlier C3s had a mechanical gauge	Dark blue	~0 Ω at 0 psi → ~90 Ω at 80 psi
Coolant temperature	NTC thermistor	Intake manifold, grounded through its threads	Dark green	Resistance falls as temperature rises; strongly nonlinear

There is no voltage sender. Corvettes before 1976 have an **ammeter** in series with the charging circuit, which we never touch; system voltage is measured directly from the supply through a divider. The wiring notes are emphatic about the ammeter for a reason: putting anything in series with it, or shorting across it, involves the full alternator current.

The piggyback tap

We do not cut the sender wire and we do not remove the gauge. We connect one high-impedance
wire to the node between gauge and sender and measure its voltage. The factory gauge still supplies the current, the needle still works, and the node voltage is a monotonic function of the sender resistance: higher resistance, more of the 12 V appears across the sender, higher node voltage.

Why measure voltage rather than resistance? Because measuring resistance requires pushing a
known current through the sender, and the factory gauge is already doing that with an unknown one. Two sources fighting over one resistor gives a reading that depends on both, and it disturbs the factory gauge. Listening to the node is passive.

The cost is that the reading depends on the gauge's internal resistance and the supply voltage as well as the sender. That is why **calibration is empirical** (below) and why the lookup goes from raw ADC counts straight to engineering units without an intermediate "resistance" step. Nothing in the chain pretends to know the gauge's resistance.

Tap points in the car are the gauge connectors behind the dash (tan, dark blue, dark green) or the bulkhead connector cavities 3 (temp) and 10 (oil). Temporary taps at tier 6 are insulation-piercing; the permanent install replaces them with soldered pigtails at the gauge terminals (spec §3.2).

Differential measurement, and why two ADCs

Doc 02 explained that the block and the console can disagree about ground by 100–200 mV. The node voltage we care about is measured *relative to the sender's ground*. If the ADC measured it relative to the console's ground instead, the ground offset would be added to every reading, and it would change with the headlights.

An ADC input can be **single-ended** (voltage between one pin and the chip's ground) or **differential** (voltage between two pins). We run each sender channel differentially, with the minus input on a second wire to the sender's own ground: the intake manifold for coolant, the block for oil, the tank strap for fuel. The offset then cancels.

The ADS1115 has four input pins that can be either four single-ended channels or two differential pairs. Three differential senders plus one single-ended voltage channel therefore need two chips:

Chip	I2C address	Pins	Channel
A	0x48	A0 − A1	Fuel
A	0x48	A2 − A3	Oil pressure
B	0x49	A0 − A1	Coolant
B	0x49	A2 single-ended	System voltage via 47k/10k divider

The ADDR pin strapped to ground gives 0x48; to VDD gives 0x49. That is the whole difference between the two boards.

ADC vocabulary

An analog-to-digital converter turns a voltage into an integer.

- **Resolution.** 16 bits means the full-scale range is split into 65,536 steps, of which a differential reading uses ±32,767. The project's `raw` values are these counts.
- **PGA (programmable gain amplifier).** The ADS1115 lets you choose what voltage equals full
  scale: ±6.144, ±4.096, ±2.048 V and so on. The spec uses ±4.096 V, so one count is 125 µV.
  Choosing a range does not raise the pin's safe limit: the chip runs from 3.3 V and no input may exceed about 3.6 V regardless of PGA. The bench doc repeats this warning because it is
  the easiest way to kill a board.
- **Sample rate (SPS).** 8 to 860 samples per second. Slower is quieter. The spec originally said 8 SPS; the firmware runs 128 SPS because at 8 SPS a single conversion takes 125 ms and
  four channels with five samples each cannot fit in a 100 ms frame. The noise budget moved to
  the digital filter instead (recorded in spec §3.3 revision 2).
- **Single-shot versus continuous.** The firmware requests one conversion, waits ~8 ms, reads
  it, and moves to the next channel. Both chips convert in parallel, so the tick fits.
- **Why not the ESP32's own ADC?** It has one, 12-bit, and it is known to be noisy and nonlinear near the ends of its range, with no differential mode worth the name. An external 16-bit converter with a real PGA and differential inputs costs $4 and removes the argument.

The front end again, with numbers

Per channel (doc 02): 10 kΩ series, 100 nF to the channel's local ground, BAT54S clamps to the
rails. The volt channel adds the 47k/10k divider before that. On the bench, pots wired as dividers across 3.3 V produce 0–3.3 V at the wiper, which through the front end lands in the ADC's range. In the car the node voltages will be somewhere in 0–12 V depending on the gauge,
which is why the spec adds "then a divider placing worst-case voltage inside the PGA range" per channel, with values to be chosen once the bench has measured real nodes.

Detecting a broken sender

With a piggyback read, a disconnected sender lets the node float up to the gauge supply, so "open" reads as a count above 97 % of full scale. A shorted sender pulls the node to ground. The firmware checks raw counts against a per-channel window *before* conversion (spec §3.4):

- Fuel, oil, coolant: above 97 % → `SENDER_OPEN_<ch>`.
- Coolant only: below 1 % → `SENDER_SHORT_CLT`. Fuel reads 0 Ω at empty and oil about 1 Ω at
  0 psi, so for them a short is indistinguishable from the low end, and 0 psi already raises `OIL_LOW`, which is the safe direction.
- Voltage: no window; `VOLT_LOW` covers it.
- A dead I2C bus is reported as every sender open, never as a frozen value.

A channel with a sender fault contributes no value fault, and the gauge shows "SENDER OPEN"
rather than a number. The percentages are starting points until the bench sweep replaces them.

Filtering: median, then IIR, then lookup

Every 100 ms the firmware takes five conversions per channel and does, in order:

1. **Median of 5.** Sort, take the middle. An ignition spike that lands on one sample is discarded entirely rather than averaged in. Medians reject outliers; means smear them.
2. **First-order IIR low-pass.** `y += α (x − y)`, one multiply-add, with α derived from a per-channel time constant τ: after one τ the output has moved 63 % of the way to a step change; after 3τ, 95 %. The constants are the point:
   - **Fuel τ ≈ 10 s.** Fuel sloshes in the tank on every corner. The gauge should show the average.
   - **Coolant τ ≈ 2 s.** Temperature changes slowly; anything faster is noise.

- **Oil τ ≈ 200 ms.** Oil pressure drops fast when something is wrong, and *seeing it drop* is the whole point of the gauge. `AGENTS.md` says "do not unify them" for this reason: a slow oil decay filtered into invisibility is the failure that destroys the engine.

3. **Lookup to engineering units.** Linear interpolation between calibration points, with linear extrapolation beyond the ends (`util::LinearCurve`, shared by firmware and host).

The filter runs on raw counts and the lookup on the filtered count, because the coolant lookup is nonlinear and filtering after a nonlinear map biases the result.

Why coolant is the awkward one

An NTC thermistor's resistance falls roughly exponentially with temperature. Two hypotheses are in the documents: the spec's original ~1365 Ω at 100 °F → ~55 Ω at 250 °F, and a C3 troubleshooting table with ~410 Ω at 100 °F and 83.5 Ω at 210 °F. They differ by a factor of three at the cold end. Both are hypotheses; the hotplate sweep in spec §9.2 decides. The practical consequence: the node voltage moves a lot between cold and warm and very little between hot and overheating, exactly where you need resolution. A 16-bit ADC helps; the calibration table having dense points at the hot end helps more.

Calibration: never fabricated

Safety invariant 5 says the shipped `config/calibration.json` is an explicitly marked placeholder and the console says so at every start. The real curves come from spec §9.2:

- **Coolant**: spare sender in a pot on a hotplate, K-type thermocouple beside it, record ADC counts every 20 °F from 70 °F to boiling. Above 212 °F is extrapolation; set the overheat alarm conservatively.
- **Oil**: spare sender on a hand pump with a mechanical gauge teed in, every 10 psi 0–80.
- **Fuel**: sweep the float arm on the bench, then correct in the car with measured 2-gallon fills, because tank geometry versus float angle is not linear and nothing on a bench captures it.
- **Voltage**: two points against a calibrated meter.

The file format holds 2–24 strictly increasing `(raw, value)` points per channel; the console refuses a file claiming `"placeholder": false` unless every channel has at least 8. On the MCU, `cal add` stages points and `cal save` activates a curve only after it has been persisted to flash, per channel, so a half-finished calibration can never become the live one.

What is confirmed and what is not

Confirmed from the 1975 factory diagram: wire colours, bulkhead cavities, the electric oil sender, the ammeter. **Hypothesis** until measured on this car: every resistance figure above, the 97 %/1 % sender windows, the divider values needed in-car, and whether the installed

senders are GM-spec or aftermarket. The engine (L48 or L82) is itself unconfirmed and does not
change the sensor plan.

---

4. The Microcontroller

What a microcontroller is, for someone who has only written for an OS

A microcontroller (MCU) is a CPU, RAM, flash and a set of peripherals on one chip, with no operating system unless you bring one. Your program *is* the machine: `setup()` runs once, then `loop()` runs forever, and when the loop is not executing anything, nothing is executing. There is no process, no scheduler you did not ask for, no virtual memory, no file system unless you add one, and no `malloc` you should trust. Reset to first instruction takes milliseconds.

The peripherals are the reason you use one. They are hardware blocks that do a job without the CPU's attention and are programmed through registers:

Peripheral	What it is	Used here for
GPIO	A pin you set to 0 V or 3.3 V, or read	Relay drives, warning LED, buzzer transistor, reverse sense
UART	A serial port: shift bytes out on TX, in on RX at an agreed rate	The link to the Pi (UART1); USB logging (UART0)
I2C	Two-wire bus controller	Both ADS1115 converters
Timers	Count clock ticks, raise interrupts	Pacing the 10 Hz tick (`millis()` under the hood)
Flash / NVS	Non-volatile storage with a key-value layer	Persisted calibration curves

The Raspberry Pi has GPIO, UART and I2C too. The difference is not the peripherals; it is that on the Pi your code runs *under* Linux and gets the CPU when Linux says so. Doc 01 lists why that is disqualifying for the warning path.

Why the ESP32-S3

The spec allowed an ESP32-S3 or a Teensy 4.0. The ESP32-S3 won for reasons worth knowing:

- **Two 240 MHz cores and 512 KB of RAM.** Enormous for this job (the firmware uses 25 KB of
  RAM and 368 KB of flash). Headroom means never having to be clever.
- **FreeRTOS underneath.** The Espressif SDK (ESP-IDF) runs a real-time OS whether you ask for
  it or not, and the Arduino layer sits on top. This gives the firmware a second task for flash writes (below) without hand-rolled concurrency.
- **Peripherals to spare.** Three UARTs, two I2C controllers, dozens of GPIOs. Wi-Fi and Bluetooth are on the chip but unused; the radios stay off.
- **Ecosystem.** PlatformIO builds it from the command line with a pinned toolchain, the DevKitC-1 board costs $15, and the Arduino API (`digitalWrite` , `Wire` , `Serial1`) keeps the
  hardware shell small enough to read in one sitting.
- **C++20.** The firmware compiles the Pi's own `protocol.cpp` , `fault_codes.cpp` and `crc16.h` unchanged, so there is one source of truth for the wire format. That needed a toolchain with `<span>` and `<charconv>` , which is why `platformio.ini` points at the pioarduino platform (GCC 13) instead of the registry's espressif32 (GCC 8.4).

"Arduino" here means the framework and API, not the 8-bit Uno board. It is a thin C++ hardware abstraction; nothing about it limits the firmware.

Pin choices are not arbitrary

`firmware/esp32/include/pins.h` avoids several GPIOs on purpose, and the reasons generalise:

- **Strapping pins** (GPIO0, 3, 45, 46) are sampled at reset to choose boot mode and voltage. Driving them from external circuitry changes how the chip boots.
- **Flash and PSRAM pins** (GPIO26–37 on the N8R8 module) are the chip's own memory bus.
- **USB pins** (GPIO19, 20) are the native USB port.
- **UART0** (GPIO43, 44) is the USB-serial console used for logs and flashing, so telemetry goes out on **UART1** (GPIO17 TX, GPIO18 RX).

The remaining assignments are bench assignments, to be frozen only after the relay board and carrier PCB confirm them.

How the firmware is structured

```
firmware/esp32/
  lib/c3fw/src/c3fw/     pure logic, no Arduino: median.h, filter.h,
lookup.h, calibration.h,
```

```
                        sender_check.h, faults.h, relays.h, command.*,
frame_writer.*
  src/main.cpp          the hardware shell: setup(), loop(), tick(), the
save task
  src/ads1115.*         I2C driver for the converter
  include/pins.h        the pin map
```

The split is the same discipline as the Pi's seams: everything that can be tested without hardware is in `c3fw` and compiled into the host's Catch2 tests by the top-level CMake (`c3_firmware` target). `main.cpp` is the only file that calls `digitalWrite` or `Wire`. If you are used to hexagonal architecture, `c3fw` is the domain and `main.cpp` is the adapter.

The 10 Hz tick, step by step

Every 100 ms `tick()` does the following, in order; command handling waits until it finishes:

1. **Sample.** Five conversions per channel at 128 SPS, both ADS1115s converting in parallel (fuel with coolant, then oil with volts). About 90 ms of the 100 ms budget. Any I2C error marks the whole tick failed and every channel reads full-scale, which the next step reports as "sender open" rather than a frozen number.
2. **Median of 5** per channel, on raw counts.
3. **Sender window check** on the median (open above 97 %, coolant short below 1 %).
4. **IIR filter** on raw counts with the per-channel τ. The class takes τ and `dt`, not a fixed α, so the response is the same if the tick cadence ever changes.
5. **Lookup** through the active calibration curve to engineering units.
6. **Fault evaluation with hysteresis** (below).
7. **Warning outputs**: LED and buzzer GPIOs set from the fault evaluator's `Warning`.
8. **Frame**: serialise `{"t":..., "fuel":..., ..., "faults":[...], "raw":{...}, "relays":{...}, "rev":..., "crc":"...."}` into a fixed buffer and queue it for UART1.

Between ticks, `loop()` services the UART: reads up to 64 available command bytes, parses complete lines, applies and acknowledges commands, and writes at most 64 bytes of queued output. It checks whether a tick is due before doing command work, so a flood of commands cannot delay sampling. A fixed eight-line transmit queue drops on overload and counts the drop.

Hysteresis, and why the buzzer has two rules

A threshold with a single value chatters: a reading hovering at 12.0 psi flips the fault on and off every tick and the buzzer stutters. Each fault therefore sets at one level and clears at a safer one (spec §4.3): `OIL_LOW` sets below 12 psi and clears at 15; `CLT_HIGH` sets above 235 °F and clears at 228; and so on for fuel and voltage.

The warning outputs derive from two faults only:

```
led    = oil_low || clt_high
buzzer = clt_high || (oil_low && volts >= 13.2)
```

The voltage gate is a piece of automotive knowledge embedded in code: with the key on and the
engine off there is no oil pressure and the system sits at 12.6 V, which is normal and should be silent. Above 13.2 V the alternator is charging, so the engine is running, so 0 psi is an emergency. Coolant over 235 °F is an emergency regardless. The LED lights either way so the condition is visible at key-on.

The warning path is local, and the code proves it

Safety invariant 2 says the buzzer must fire with the Pi powered off. Structurally: the evaluator that fills `faults[]` for the frame is the same object whose `warning()` drives the GPIOs, in the same tick, before the frame is queued. Nothing in that path reads from the UART, waits on the transmit queue, or checks whether a Pi has ever answered. The bench acceptance test is literal: unplug the serial adapter with the oil pot at zero and watch the LED and buzzer keep doing exactly what they were doing.

Relays: commanded state, held state

The MCU keeps a `CommandedState` (blower 0–4, A/C, mode, headlights, doors) and derives the
nine output pins from it with `outputs_for()`. Rules enforced in code, not on the Pi:

- Blower speed energises exactly one of four tap relays; 0 energises none. Taps are switched break-before-make so two resistor-pack taps are never bridged.
- The A/C clutch is refused while the blower is off (the evaporator would freeze).
- Mode is two vacuum solenoids: floor energises one, defrost the other, vent releases both.
- Outputs change only on a command and hold otherwise. **Losing the Pi does not turn the blower off** (invariant 6). This is hold-last-state while the MCU runs; on MCU reset everything initialises off, which is why defrost also needs a mechanical path (doc 07).

Every frame re-asserts the full relay state so the Pi's dock displays what the MCU is actually doing, not what the user last tapped.

Flash writes on a second task

Writing to flash on the ESP32 stalls the CPU for tens of milliseconds while the cache is disabled. Doing that inside `tick()` would blow the sampling deadline and delay a warning. So `cal save` hands the curve to a FreeRTOS task through a fixed-size mailbox; the task writes

NVS (`Preferences`) and posts a result; the main loop activates the new curve and sends the acknowledgement only after the write succeeded. A failed save leaves the previous curve active and the staged points intact. Mailboxes rather than `std::atomic` structs because the ESP32 libstdc++ does not guarantee lock-free atomics for anything wider than a word.

Flashing, monitoring, first boot

```
cd firmware/esp32
pio run                  # build
pio run –t upload        # flash over the DevKit's UART USB port
pio device monitor       # UART0 logs at 115200
```

First line names the build and reports `adc 0x48 ok, 0x49 ok` and `calibration PLACEHOLDER`.
`MISSING` on an ADC means an I2C wire or the ADDR strap. If upload cannot find the board: hold
BOOT, tap RESET, release BOOT, retry. The DevKit has two USB-C ports; the one marked UART is
the console.

Not yet proven

The firmware compiles and its pure logic passes host tests. Nothing has been flashed to a board. Spec §11 still requires measuring warning timing under a UART flood, behaviour during NVS writes, and I2C fault handling on real hardware. Until then, statements about MCU timing are design intent.

---

5. The Wire Between Them: UART, Framing & Failure

UART from first principles

A UART (universal asynchronous receiver/transmitter) sends bytes one bit at a time on a single wire, with no clock wire. Both ends agree in advance on the bit rate (the *baud* rate, 115200 bits per second here), and each byte is wrapped in a start bit and a stop bit so the receiver can find byte boundaries. "8N1" means 8 data bits, no parity bit, 1 stop bit: ten bits on the wire per byte.

Three wires: the MCU's TX to the Pi's RX, the MCU's RX to the Pi's TX, and a shared ground. Both sides are 3.3 V logic, so no level shifter is needed between the ESP32 and the Pi, and the bench adapter to a Mac must be a 3.3 V one.

Bandwidth: `115200 / 10 = 11,520 bytes/s`. A telemetry frame is around 250 bytes, so 10 Hz uses roughly a quarter of the link. Ample, and slow enough that the wiring is forgiving.

What UART does *not* do: no addressing (it is point to point), no error detection, no flow control unless you add wires, and no guarantee that the two ends agree on where a message starts after a glitch. Everything above the byte level is ours to define.

Why UART and not something else

Option	Why not here
USB (CDC serial)	Needs the Pi's USB stack up, which is late in boot and shared with the dongle, camera and touch. A UART is a pin on the Pi's SoC, alive from the kernel's first seconds.
I2C or SPI between boards	Designed for centimetres on one board, with one side strictly in charge. Poor over a cable in a noisy car, and the Pi would have to poll.
CAN	The right answer in a modern car with many modules. Here there are two boards and a 1975 car with no CAN anywhere; it would add transceivers, a stack, and a message-ID design for no benefit.
Ethernet/Wi-Fi	Far more machinery than the problem, and a radio in the console is a debugging and EMI liability.

Doc 11 has the short version for conversation.

Framing: newline-delimited JSON

Each message is one line of JSON ending in `\n`. The choice is deliberate and worth defending:

- **Self-synchronising.** After noise, the receiver discards bytes until the next newline and is back in sync. A binary format needs a sync word and a length field to do the same.
- **Human-readable.** `pio device monitor`, `cat /dev/ttyAMA0`, a text editor, and a CSV file all show the truth. On a car with a fifty-year-old harness, being able to read the wire with no tools is worth more than the bytes saved.
- **Extensible without versioning.** Unknown keys are skipped, so a field can be added on the MCU before the Pi learns it (spec §4.1 makes `raw`, `relays` and `rev` optional for exactly this reason).
- **Cheap enough.** At 10 Hz, parsing cost is irrelevant even on the MCU.

The parser is hand-rolled (`telemetry/protocol.cpp`): fixed buffer, no allocation, no exceptions, returns `std::optional<Frame>`. nlohmann/json is used only at startup for config and calibration files. Lines over 512 bytes are dropped.

A frame:

```
{"t":184523,"fuel":0.62,"oil":41.2,"clt":196.4,"volt":14.1,"faults":[],
 "raw":{"fuel":12843,"oil":8211,"clt":19004,"volt":21877},
 "relays":
{"blower":2,"ac":true,"mode":"vent","headlights":false,"doors":false},
 "rev":false,"crc":"a3f1"}
```

`t` is MCU milliseconds since boot; the Pi never trusts it for freshness (below). Values are already in engineering units; the Pi sees raw counts only in the calibration screen.

The CRC, and why a checksum was not enough

A cyclic redundancy check is a 16-bit polynomial hash of the message appended by the sender and recomputed by the receiver. Any mismatch means the line was corrupted in transit and is discarded. A plain sum would let two flipped bits cancel; a CRC-16 catches all single- and double-bit errors and any burst up to 16 bits, which is the shape of UART noise.

The project uses **CRC-16/CCITT-FALSE**: polynomial 0x1021, initial value 0xFFFF, no reflection, no final XOR, check value `crc("123456789") == 0x29B1`. Naming the variant precisely matters because there are half a dozen "CRC-16/CCITT" variants that disagree, and the check value is how you know two implementations match. The CRC covers the line exactly as
the MCU serialised it with the `,"crc":"xxxx"` member removed and the closing brace kept, and
`crc` is always the last member. Firmware and host compile the same `crc16.h`.

Bad CRC is counted separately from malformed JSON so a bench log distinguishes noise from a
firmware bug. Neither kind of line is ever rendered.

Plausibility limits are not thresholds

The Pi rejects a frame as corrupt when fuel is outside 0–1, oil outside 0–200 psi, coolant outside −40–400 °F, or volts outside 0–40 V. These catch a frame whose CRC happened to pass
but whose content is nonsense. They are deliberately far outside any real operating range so they can never be confused with a warning decision. The Pi has no code that compares a value to 12 psi or 235 °F; it renders the `faults` array. Keeping this distinction crisp is how the codebase honours invariant 1 while still validating input.

Freshness is measured on receipt

The `t` field comes from the MCU's clock, which is unsynchronised with the Pi's. The Pi stamps each *accepted* frame with its own monotonic receive time and greys the gauges when the newest
accepted frame is older than `stall_timeout_ms` (1–1000 ms, default 1000). Invalid frames do not refresh the deadline. This is safety invariant 4: a frozen needle is worse than no needle.

Commands and acknowledgements

The Pi sends commands as single lines:

```
{"cmd":"hvac","blower":2,"ac":true,"mode":"vent"}
{"cmd":"lamp","headlights":true,"doors":false}
{"cmd":"cal","ch":"fuel","action":"add","raw":12843,"value":0.50}
{"cmd":"cal","ch":"fuel","action":"save"}
{"cmd":"ping"}
```

The MCU answers every one with `{"ack":"hvac","ok":true}` or `{"ack":"hvac","ok":false,"err":"blower must be 0..4"}`. Rules that matter:

- **Whole-message validation.** `hvac` needs all three fields; a missing or malformed field rejects the command entirely so a relay never moves on half a message.
- **Requests, not state.** A tap on the Pi submits intent; the dock keeps displaying the MCU's reported relay state from the next frame. Confirmation requires an `ok:true` ack of the same kind *and* matching relay telemetry afterwards. The UI shows SENDING, WAITING FOR
  STATE, or NO CONFIRMATION explicitly.
- **One transaction at a time, 1 s timeout.** The protocol has no transaction identifiers, so a late ack could otherwise be mistaken for the answer to a newer command. After a timeout the host's command session refuses further submissions until the process restarts. That is conservative on purpose; designing coordinated recovery is an open item.
- **Unknown door state disables lamp changes.** A `lamp` command carries both `headlights`
  and `doors`; if the MCU's frames have not reported door state, the Pi will not send a command that would overwrite it with a guess.

Failure behaviour, both directions

Situation	MCU	Pi
No command for 5 s	Holds relay state. The blower does not fail to off.	—

Situation	MCU	Pi
No valid frame for 1 s	—	Gauges grey out, "NO DATA", comms banner; HVAC dock shows unknown
Bad CRC / malformed line	Counted, ignored	Counted, dropped, never rendered
UART cable unplugged	Warning LED and buzzer unaffected	Link reports Failed immediately
Pi crashes or reboots	Unaffected	Watchdog reboots it (doc 06)

Recording and replay fall out of the design

Because every frame is a self-contained line with `raw` and converted values, recording is appending rows to a CSV and replay is reading them back paced on the `t` deltas. A stall on the road replays as a stall on the desk. Gaps over 10 s are shortened to 10 s; after the last row the link reports Failed so the stale path is exercised. Raw counts in the same row as converted values mean a calibration change can be re-evaluated against old drives without driving them again. `--record=PATH` works with any source; `--replay=PATH` is the tier-4 rung of the simulation ladder.

Where the code is

`src/telemetry/protocol.*` (parser, shared with firmware), `crc16.h`, `uart_source.*` (owns the descriptor in both directions, non-blocking, in-order frames), `vehicle/command.*` (command formatting and ack parsing), `app/telemetry_ingestor.h` (records then publishes), `firmware/esp32/lib/c3fw/frame_writer.*` and `command.*` (MCU side).

---

6. The Display Host: Linux as an Appliance

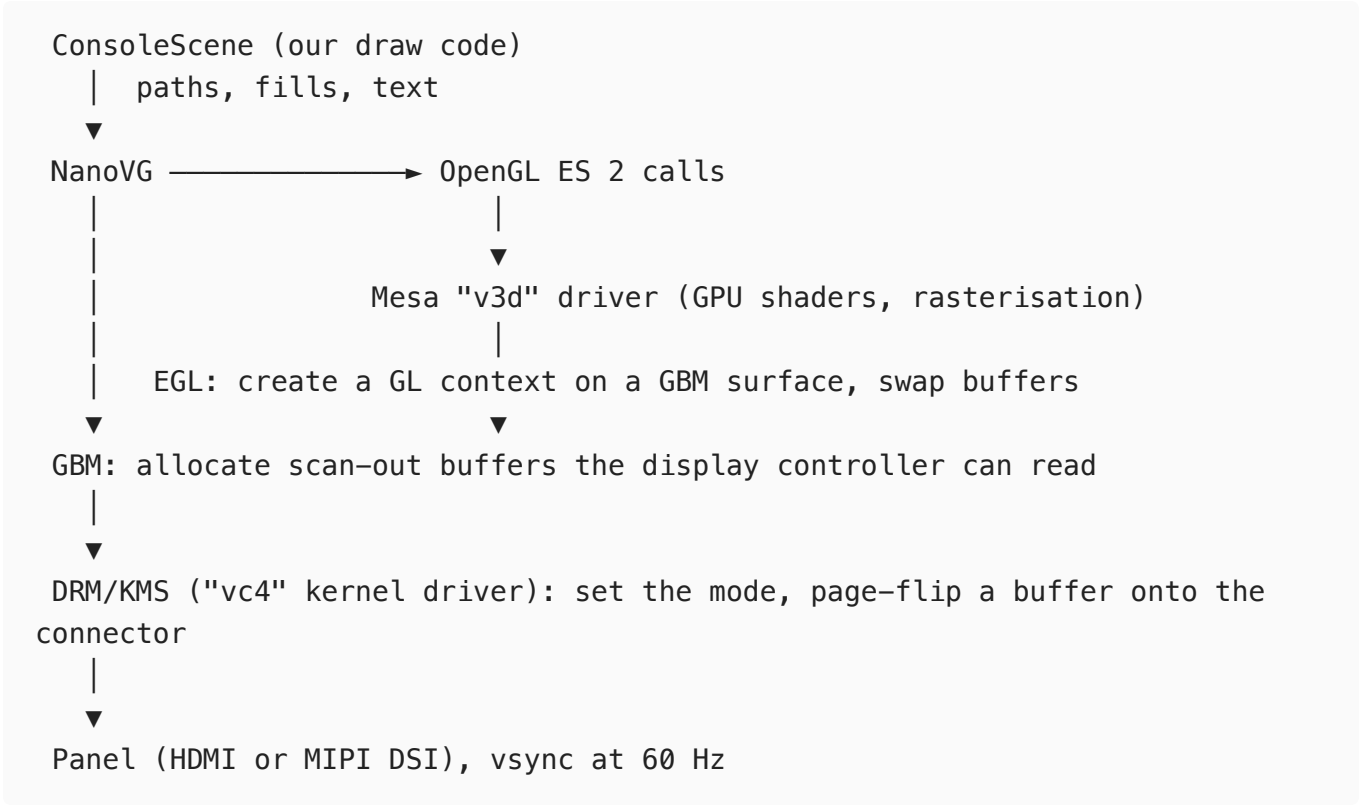
The Pi runs Linux, but not the Linux you use on a laptop. There is no desktop, no window manager, no display server, no login. One process owns the screen from boot to ignition-off. This note explains the stack it sits on and the rules it follows to be trustworthy in a car.

Why a CM4, and why not a Pi 5

The Compute Module 4 is the Raspberry Pi 4's silicon (BCM2711: four Cortex-A72 cores, VideoCore VI GPU) on a small board with eMMC storage and no connectors, meant to be mounted on
a carrier board you choose. Reasons it beat the alternatives:

- **Hardware H.264 decode.** CarPlay video arrives as H.264. The BCM2711 decodes 1080p in a
  dedicated block; the Pi 5's BCM2712 dropped that block (it kept HEVC) and decodes H.264 in
  software, which is faster on paper and hotter in a closed fibreglass cabin in August. The decoded frame also comes out as a DMA-BUF the GPU can sample without copying (below).
- **eMMC, not an SD card.** Soldered flash survives vibration, heat and power loss better than a card in a socket, and the spec forbids SD.
- **Carrier flexibility.** A CM4 carrier can expose exactly the ports needed (USB host for dongle, touch, camera, mic; MIPI DSI or HDMI for the panel; the UART pins) and nothing else.
- **A supported 64-bit Debian.** Raspberry Pi OS Lite Bookworm, GCC 12, C++20, mainline kernel graphics drivers. The build container in `tools/docker/` is exactly this image, so what passes there passes on the module.

The graphics stack, from our code down to photons



The acronyms, one line each:

- **DRM** (Direct Rendering Manager) is the Linux kernel subsystem that owns GPUs and display
  controllers. `/dev/dri/card0`.

- **KMS** (Kernel Mode Setting) is the part of DRM that picks a resolution and refresh rate on a *connector* (the physical output), through a *CRTC* (the scan-out engine), and swaps the *framebuffer* it is scanning with a *page flip* that happens at vsync.
- **GBM** (Generic Buffer Management) allocates buffers in a format the display controller can scan out (`XRGB8888` here) and hands them to EGL as a "native window".
- **EGL** is the glue between OpenGL ES and whatever windowing system exists. On a desktop that is X or Wayland; here it is GBM directly, which is exactly how Weston and Kodi do it.
- **GLES2** is the OpenGL ES 2.0 API NanoVG draws with on the target (`nvg_gles2.c`); the desktop uses GL3 (`nvg_gl3.c`). Same NanoVG, different backend file.

`DrmBackend` does all of this: open the card, find a connected connector and its preferred mode, create a GBM surface, an EGL context with an 8-bit stencil (NanoVG needs it for antialiased fills), set the CRTC once, then page-flip every frame and wait for the flip event (100 ms bound). On exit it restores the CRTC it found. Most panels scan out landscape, so `[display] rotation` rotates drawing and touch identically and scene code always sees portrait.

Why no X or Wayland. They add a second process that owns the screen, a compositor that adds a frame of latency, and hundreds of megabytes of packages that can break on update. An appliance with one full-screen client gains nothing from them. Kiosk products, TVs and car head units skip them for the same reason.

Input

Touch arrives as a USB HID device. The kernel exposes it as an evdev node (`/dev/input/eventN`);
libinput turns raw events into touch-down, motion and touch-up with contact IDs, and **libudev** finds the devices on seat0 and notices hot-plug. `LibinputSource` reads one finger, rotates the coordinates with the display, and delivers them to the same `InputSource` interface the SDL mouse feeds on the desktop. Gesture ownership is captured on touch-down: a
finger that starts in the CarPlay well stays with CarPlay even if it drags out, and a surface takeover (reverse gear engaging) sends a cancellation rather than a release.

The event loop is a game loop with descriptors

One thread waits in `poll(2)` on a set of file descriptors (UART now; libusb, libinput and the DRM fd later) with a timeout computed to the next 60 Hz tick. Readable descriptors are serviced; when the tick is due, the loop ingests pending telemetry, updates models, draws the scene, presents, and kicks the watchdog. Pacing has no catch-up bursts: a late tick is followed by the next one immediately, never by several back to back.

The spec originally said epoll plus timerfd. `poll` was chosen because at five descriptors they are indistinguishable and `poll` also exists on macOS, where the desktop build runs. Recorded as a deviation.

Soft real-time: why no heap, no exceptions, no locks in the render path

The Pi is not hard real-time (Linux can always preempt us), but the render loop has a budget of 16 ms and the watchdog is watching. Three rules keep the worst case bounded:

- **No heap allocation after init.** `malloc` takes a lock inside libc and can take a page fault; neither has a bounded time. Everything is preallocated: `util::fixed_vector`, fixed text buffers, fixed queues. (NanoVG's own caches and the GPU driver are not yet certified allocation-free; that is an open item for target profiling.)
- **No exceptions.** Unwinding is slow and unpredictable, and a throw across the render loop leaves the display in an unknown state. Errors are return values (`std::optional`, result structs, `PresentResult`). Exceptions are allowed at startup, where failing is fine.
- **No locks.** A mutex the render thread waits on turns another thread's hiccup into a missed frame. State crosses threads through the triple buffer.

The triple buffer

One producer, one consumer, three slots. At any instant one slot is being written, one is being read, and one is parked in the middle; ownership changes hands through a single atomic exchange. The producer always publishes into its private slot and swaps it to the middle; the consumer swaps the middle into its private slot. Nobody reads a slot someone else is writing, no lock, no waiting, and the consumer always sees the newest value, never a queue. Two slots cannot give this guarantee: the producer can wrap around onto the slot the consumer is still copying. `util/triple_buffer.h` requires `T` to be trivially copyable so a snapshot is a `memcpy`, never a shared reference.

Where the loop needs *every* item rather than the latest (CSV rows, log lines), it uses a bounded SPSC queue instead, and drops on overflow with a count. Choosing between "latest wins" and "ordered, may drop" is a decision made per data stream, not a default.

Background I/O

Writing a CSV row, saving settings, printing to stderr and converting local time all touch the file system or libc in ways with unbounded latency. `BackgroundIo` owns them on a worker thread fed by fixed-capacity, non-blocking queues (256 CSV frames, 128 log lines). Settings use a latest-value triple buffer so rapid taps coalesce. Shutdown joins the worker and drains accepted work. Sudden power loss can lose queued records; the design accepts that rather

than
block the render loop on `fsync`.

The watchdog

`bcm2835_wdt` is a hardware timer in the SoC exposed as `/dev/watchdog`. Once opened, it must
be written to ("kicked") periodically or it reboots the board. The render loop kicks it only after a tick that *successfully presented* a frame, so a hung GPU, a stalled loop or a deadlock all lead to a reboot within seconds rather than a frozen screen. On a deliberate exit the loop writes the magic character `V` before closing so the driver knows the stop was intentional. A failed present stops the loop without a further kick. Off by default on the desktop; enabled with `[platform] watchdog_device` on the target.

Read-only root, writable data

Spec §5.6: the root filesystem is mounted read-only with an overlayfs on top, `/var/log` goes to tmpfs, journald is volatile, no swap. Power loss mid-write is the classic way an SD card Pi dies, and a read-only root removes the whole failure class. The two files the console must write, `settings.toml` and later an in-car `calibration.json`, live on a small separate partition at `/data`, written atomically (temp file, `fsync`, rename, directory `fsync`). A missing or corrupt settings file falls back to defaults and says so; it never stops the console. A systemd unit with `Restart=always` brings the process back after a crash.

Boot and shutdown as a timeline

```
 key to ACC ──► master relay closes ──► DC-DC up ──► kernel ──► systemd ──►
c3console
   0 s                                              ~3 s
<10 s: first gauge frame

+10–20 s: CarPlay session (phone-side, not ours)
 key off ──► ignition-sense opto drops ──► finish frame, fsync /data, halt
──► supercap drains
                                                2–3 s budget
```

The gauges must be drawn before CarPlay is negotiated, so the well shows a connecting state while the dongle talks to the phone. The ignition-sense GPIO handler is not yet written; the supercap and the read-only root are what make the window tolerant.

CarPlay: the path we have not built yet

The Carlinkit dongle is a USB device that speaks Apple's CarPlay protocol to the phone over Wi-Fi and Bluetooth, and speaks a reverse-engineered vendor protocol to us over USB. From us
it needs a video mode request (we ask for exactly the well's pixel size, so no scaling) and a stream of touch events in well-local coordinates; to us it sends H.264 frames and PCM audio. The references are `FastCarPlay` and `node-carplay`; `AGENTS.md` forbids inventing framing.

The decode path is the reason for the CM4: H.264 → **V4L2 M2M** (the kernel's memory-to-memory
codec interface to the hardware decoder) → **DMA-BUF** (a kernel handle to a buffer that several devices can share without copying) → **EGLImage** via
`EGL_EXT_image_dma_buf_import`
→ sampled as a GL texture inside the well while NanoVG draws gauges around it. That is spec §5.4 stage 1: zero-copy, moderate code. Stage 2, putting video on a separate DRM overlay plane
so the display controller composites it during scan-out for zero GPU cost, is explicitly deferred until stage 1 has run in the car.

Touch routing is a rectangle test: a finger inside the well is forwarded to the dongle in local coordinates; outside it is ours. CarPlay never knows it lives in a sub-rectangle.

Audio

The Pi outputs digital audio over **SPDIF** on a HiFiBerry Digi+ Pro to a DSP amplifier with an optical input. Optical means no copper between console and amplifier, so no ground loop and
no alternator whine, a whole class of problems removed before it starts on a fibreglass car with an unshielded high-energy ignition a few feet away. ALSA is the kernel audio API; the volume, balance, tone and preset
controls in the audio sheet become a small Pi-side DSP stage before SPDIF. A USB sound card takes an electret microphone for Siri and calls. None of this is implemented yet; `AudioSink` is the seam it will plug into.

The rear camera

A USB UVC camera, imported through the same V4L2 → DMA-BUF → texture path as CarPlay, shown
in the well when the MCU's frame says `rev:true`, held 2 s after it clears, with NanoVG guidelines on top. If frames stop, the surface says "Camera unavailable, use mirrors" rather than freezing, invariant 4 applied to video.

The desktop build is the same program

`DesktopBackend` (SDL2 window, GL 3 context) and `SdlInput` implement the same seams, and
`MockSource` runs the physics profiles from the browser mockup. Everything above the seams (scene, controllers, ingestion, settings, event loop, triple buffer) is identical on both. Screenshots without a display work in the container with `SDL_VIDEODRIVER=offscreen`. This is the reason the project could reach this point with no hardware in hand.

---

7. Controlling the Car: HVAC, Lamps, Doors, Reverse

Reading the car is passive. Controlling it means closing contacts on circuits that already have switches, so this note starts with how the factory circuits work and then shows where a relay goes so the factory switch keeps working.

How the factory heater and A/C work

Four separate mechanisms, all from the 1960s:

1. **Blower speed is a resistor pack.** The blower motor runs from 12 V. The dash switch routes the motor's feed through zero, one, or several series resistors mounted in the heater box (where the airflow cools them). Each switch position is a *tap* on that resistor ladder; high speed bypasses the resistors, on A/C cars through a dedicated high-blower relay. Motor current is 15–20 A.
2. **Mode doors are vacuum.** Engine vacuum, stored in a reservoir, moves diaphragm actuators
   that swing the doors between vent, floor and defrost. The dash lever operates a vacuum switching valve that routes vacuum to the right actuator.
3. **Temperature blend is a cable.** The lever pulls a Bowden cable that moves the blend door over the heater core. Purely mechanical.
4. **A/C compressor clutch** (if the car has C60 air) is an electromagnetic clutch fed from the dash switch *through a low-pressure cut-out switch*, so the compressor cannot run with no refrigerant.

The screen covers the factory control panel, so it must offer every function the panel did. Spec §6 is on the critical path of the install for this reason alone.

The 80 % version: relays where the switch was

Function	How	Relay outputs
Blower speed 1–4	A relay per resistor tap, contacts in parallel with the switch's positions	4 (three on taps carrying motor current, one on the high relay's trigger on A/C cars)
A/C clutch	One relay in parallel with the switch contact, *upstream* of the pressure switch so the cut-out stays in the loop	1
Vent / floor / defrost	12 V 3-way vacuum solenoid valves replacing or paralleling the switching valve	2 (floor, defrost; vent releases both)
Temperature blend	**Keep the cable lever**, protruding through a slot below the screen	0

Rules the firmware enforces (doc 04): exactly one blower tap energised at a time, break-before-make so two taps are never bridged (that would short part of the resistor ladder); clutch refused with the blower off; outputs change only on command and hold on comms loss.

Why relays and not PWM. Speed-controlling a 20 A inductive motor with a transistor means a high-side MOSFET driver with gate drive, heatsinking, flyback handling and EMI filtering, a project in itself. Four relays on the existing taps reproduce the factory behaviour exactly, with parts rated for it, in an afternoon. Phase two (an electric blend door with position feedback) is explicitly optional.

Defrost must not need software

Spec §6.3 is non-negotiable: defrost is a visibility-safety function. Either the retained lever keeps a mechanical defrost position, or a hard-wired switch drives the defrost solenoid directly, bypassing the MCU. This is because "hold last state" only holds while the MCU is running; a reset initialises every output off. A touchscreen button is not a fallback.

Headlamps: the one rule with no exceptions

The factory headlight switch stays wired and authoritative. Our relay's contacts go *across* the switch's headlamp contact (red feed to dark-green output at the switch connector, per the factory diagram). Either can light the lamps; neither can stop the other.

Failure analysis, which is the way to explain the rule:

Failure	Result
Relay fails open	Factory switch still works. Fine.

Failure	Result
Relay fails closed	Lights stay on. Drains the battery when parked, never leaves you dark. Acceptable.
Pi kernel panics at night	Factory switch unaffected. Fine.
MCU resets	Relay output initialises off; factory switch unaffected. Fine.

A relay *in series* would invert every row of that table, which is why "never put the Pi or the MCU in series with the lamp circuit" is invariant 3. The dimmer (high/low beam) is a separate floor switch downstream; if a High Beams control survives the UI review, it is a second parallel relay across the dimmer's light-green output, never a replacement.

One subtlety the code respects: `headlights=false` in a frame means *our relay is off*, not that the lamps are dark. The factory switch may have them on and the MCU cannot see that. Auto theme follows our relay state for this reason and is not an ambient-light sensor.

Headlight doors are vacuum, and the valve is in the switch

On a C3 the pop-up headlight doors are vacuum actuators, and the vacuum valve that opens them
is built into the headlight switch: pull the switch, the electrical contact closes *and* the valve routes vacuum to the door actuators. Energising the lamp filaments through our relay therefore does **not** open the doors. Door control from the screen is a 12 V 3-way vacuum solenoid teed into the door circuit in parallel with the switch's own valve, driven by the door relay. The factory manual override valve under the column stays. Same shape as the lamp rule, applied to vacuum. The "R/L HDLT DOOR SOLENOID" boxes on the engine sheet are the position switches for the dash indicator lamp, not actuators; that circuit is left alone.

Reverse sense

The backup-lamp circuit (light green through bulkhead cavity 24) is +12 V only in reverse. An optocoupler's LED sits across it; the phototransistor pulls the MCU's `kReverseSense` input low. The frame carries `rev:true`, and the Pi shows the rear camera surface, holding it 2 s after reverse clears. Reverse is trusted only from a live frame; on a telemetry stall the last known gear state is kept, the same policy as HVAC.

What the relay hardware is

Automotive SPDT relays, 30 A contacts, in sockets, with flyback diodes across the coils, on a board fed from a fused 20 A branch (doc 08). Coil current ~150 mA at 12 V per relay; the MCU drives each through a transistor or an opto-isolated relay module, never directly from a GPIO.

On the bench a 5 V eight-channel opto-isolated module with the jumper set to **high-level trigger** stands in, so that an unpowered or resetting MCU means every relay off.

Verifying before wiring

Every colour above is from the 1975 factory diagram or an owner's trace, and the wiring notes say which. Before any relay is connected: confirm the heater-only versus A/C harness (the compressor and the A/C position on the dash control are the tell), meter the switch terminals (dark green is 12 V only with headlamps on; red is always live), and confirm ACC behaviour on the chosen trigger wire. The pin map in `pins.h` is provisional until the relay board and carrier PCB confirm it.

---

8. Power and Grounding

The tree

```
Battery + (behind the driver's seat)
  └─ 60 A ANL fuse, within 18" of the battery
      └─ 8 AWG ──► AUX FUSE BLOCK (behind the console)
            ├─ 10 A ──► automotive DC-DC ──► Pi 5 V rail + supercap module
            ├─ 10 A ──► ESP32 board + ADS1115 rails
            ├─ 20 A ──► relay board common (HVAC, lamp parallel relay)
            └─ 40 A ──► DSP amplifier (remote-on from Pi GPIO via opto)

MASTER RELAY (30 A) — accessory
  pin 86 ◄── trigger: factory radio ACC feed (RADIO fuse output)
  pin 85 ──► ground
  pin 30 ◄── fused 20 A from the aux block
  pin 87 ──► DC-DC enable + MCU enable

IGNITION SENSE ──► optocoupler ──► Pi GPIO (shutdown trigger)

GROUND — one star point on the steel birdcage, sanded bare, star washer,
dielectric grease
  ├─ Pi / DC-DC: 12 AWG
  ├─ MCU board:  16 AWG
  └─ DSP amp:     8 AWG
```

Power runs down one rocker panel, audio and signal down the other, so the amplifier's supply current does not induce noise into the sensor wires.

Why a separate feed from the battery

The factory fuse block is sized for a 1975 car's loads with nothing to spare, and its wiring is fifty years old. A dedicated fused feed from the battery gives the console its own protected supply, keeps our current out of the factory harness, and means removing the console removes every wire we added. The main fuse protects the 8 AWG run; each branch fuse protects its own wire (doc 02: fuses protect wire, not devices).

The master relay, and why ACC

Everything except the amplifier is switched by one relay whose coil is driven from the factory radio's accessory (ACC) feed. Consequences:

- **No parasitic draw.** When the key is out, the coil is off, and nothing downstream of the relay can draw from the battery. The Pi and MCU are hard-off, not sleeping.
- **The ignition switch carries only coil current.** A few hundred milliamps through the old switch instead of the console's several amps.
- **It behaves like the factory radio.** Live in ACC and RUN, dead in START and OFF, which is what a driver expects of a head unit. The wiring notes say to identify the ACC wire with a meter rather than trust colour conventions, because the factory diagram does not label the ignition-switch functions.

Ignition sense and graceful shutdown

ACC dropping cuts power at the relay. The Pi needs a warning before that. The ignition (IGN, pink) feed goes through an optocoupler to a Pi GPIO; when it drops, the software finishes the frame, syncs the data partition and halts, while a supercapacitor module on the 5 V rail carries the 2–3 s that takes. The relay is on ACC and the sense is on IGN, so turning the key from RUN back to ACC gives the shutdown signal while power is still present; turning straight to OFF relies on the supercap.

The GPIO handler is not written yet. The read-only root (doc 06) means an unclean cut is survivable in the meantime; it just risks a settings change that has not reached disk.

Grounding on a fibreglass car

A steel-bodied car grounds through the body. A Corvette's body is fibreglass, so every ground is a wire to the frame or the engine, and the quality of those wires shows up directly in the gauges: a poor cluster ground famously makes the factory temperature gauge read high.

The spec's answer is a **star ground**: one point on the steel birdcage (the structural frame around the cabin), sanded to bare metal, with a star washer to bite through any remaining oxide and dielectric grease to keep it that way, and one wire from each board to that point. Star rather than chain because chaining puts board A's return current through board B's ground wire, and the voltage drop along that wire becomes an error in board B's measurements.

Ground
wires are sized like the supplies they return.

The sensor channels take this one step further and do not use the console ground at all for their reference: each differential minus input runs to the sender's own ground (doc 03).

The DC-DC converter

"Automotive-rated, 9–36 V in, 5 V / 5 A out, load-dump rated" (spec §2). Each phrase is a requirement: 9 V input tolerance so the Pi stays up during cranking; 36 V and load-dump rating so an alternator transient does not reach the Pi; 5 A because a CM4 with a panel backlight, a dongle, a camera and a touch controller on its USB ports can draw 3 A or more. A TVS diode on the 12 V rail ahead of the converter adds a second layer. A phone charger from a petrol station meets none of this.

Heat

An 11" panel at full brightness plus a CM4 is 25–40 W behind a fibreglass dash in the sun. The CM4 gets an active heatsink and blower; the assembly gets vented; and the spec suggests dimming the panel on a thermal threshold rather than letting the module throttle. This is one of the "things that will bite you" in spec §12, alongside sunlight readability (hence the 1000-nit, optically bonded panel requirement, with no software fix available).

Audio stays digital

The Pi to amplifier link is optical SPDIF. There is no copper audio path between the console and the amp, so there is no ground loop and no path for alternator whine or ignition noise into the speakers. Crossovers, time alignment and room EQ live in the amplifier's DSP; the Pi does volume, balance, fader and tone. The amplifier's remote-on line is driven from a Pi GPIO through an optocoupler, so the amplifier powers up after the Pi and never sees a 3.3 V logic pin directly.

What is decided and what waits for the bench

The topology above is spec §8 and is settled. The specific DC-DC module, supercap module, TVS
part, relay sockets and PCB are Phase 3/4 purchases deliberately deferred until the bench has frozen the pin map and front-end values (bench doc, section D). Wire colours for ACC and the radio fuse are to be confirmed with a meter on the car.

---

9. Safety and Failure Thinking

Business software fails and someone reruns a job. This system fails and someone is on a dark road with no oil pressure. The habit that keeps that from mattering is asking, for every component, "what happens when this is wrong, absent, or stuck?" and making the answer boring.

The six invariants

#	Invariant	The failure it prevents	Where it is enforced
1	**The MCU owns everything safety-adjacent.** Acquisition, filtering, conversion, relay drive and the warning buzzer live in firmware. The Pi is display, CarPlay and audio.	A Linux stall, a GPU hang, a USB storm from the dongle, or a Pi crash delays or loses a warning.	Firmware `tick()` evaluates faults and drives GPIOs; the Pi's parser has plausibility limits but no thresholds; `c3_firmware` is never linked into the console binary.
2	**The warning path is local to the MCU and independent of UART.** It fires with the Pi powered off.	A design where the Pi tells the MCU to buzz, and the Pi is not there.	Same evaluator fills `faults[]` and `warning()`; no read from UART in that path; bench acceptance test: unplug the serial cable, LED and buzzer unchanged.
3	**Never in series with the headlamp circuit.** Lamp control is a relay in parallel with the factory switch, which stays authoritative.	Any failure of our hardware or software leaving the car without headlights.	Relay contacts across the switch terminals (red to dark green). Code never assumes exclusive control; `headlights=false` means "our relay is off", nothing more.
4	**Never display a stale value as live.** After 1 s without a valid frame, gauges grey out and show a comms fault.	A frozen needle reading 40 psi while the engine runs dry.	`TelemetryView` freshness against host receive time; invalid frames do not refresh it; camera surface shows "unavailable" rather than a frozen frame; `LinkLoss` mock profile exercises it on every desk.
5	**Never fabricate calibration.** Curves come from bench measurement; the shipped table is a marked placeholder.	Gauges that look precise and are wrong; an overheat alarm set from an invented curve.	`config/calibration.json` has `"placeholder": true`; the console refuses `false` without ≥ 8 points per channel; MCU `cal save` activates only after persistence; the console says PLACEHOLDER at every start.

#	Invariant	The failure it prevents	Where it is enforced
6	**HVAC holds last state on comms loss.** Never fail the blower to off. Defrost stays reachable without software.	Losing the blower on a hot day, or losing defrost in rain, because a cable came loose.	Firmware changes outputs only on command; no host timeout switches anything off; defrost has a mechanical or hard-wired path independent of the MCU.

Fail-safe, fail-operational, and knowing which you want

Two different postures show up in the table, and it is worth naming them:

- **Fail-safe** means "when in doubt, go to the safe state". The relay board is fail-safe: active-high drive, everything off on MCU reset. Sender faults are fail-safe: an open sender reads "SENDER OPEN", not a value, and a dead I2C bus reads every sender open. `OIL_LOW` at
  0 Ω is fail-safe: a shorted oil sender raises the alarm rather than hiding.
- **Fail-operational** means "keep doing the last known good thing". HVAC is fail-operational: the blower keeps its speed when the Pi disappears. The headlamp switch is the ultimate fail-operational component: it works no matter what we do.

The design is careful about which applies where. Turning the blower off on comms loss would be
"safe" for the electronics and dangerous for the driver. Holding a stale gauge value would be "operational" and dangerous. The table above is the record of those choices.

Hysteresis is a safety feature

A threshold with one value makes a reading on the line flip the fault every tick, and a stuttering buzzer gets ignored or disconnected. Every fault sets at one level and clears at a safer one (12 → 15 psi, 235 → 228 °F). The desktop mock simulates the same numbers so the UI
is exercised against realistic fault behaviour, but the production Pi executable never evaluates them.

Whole-message validation

A command with a missing field is rejected entirely (`{"cmd":"hvac","blower":3}` without `ac` and `mode` does nothing). A frame with a bad CRC or an out-of-range value is dropped entirely.

A `relays` object with a duplicate or missing member rejects the frame. Half-applied state is the failure mode being avoided: a relay that moved on a message the sender did not finish.

Requests, not writes

The UI never sets relay state. It sends a request, and the dock shows what the MCU reports in the next frame, with SENDING / WAITING FOR STATE / NO CONFIRMATION visible in between. If the
Pi's idea of the blower speed could diverge from the MCU's, one of them would be lying to the driver; re-asserting the full relay state every frame makes that impossible.

Reversibility is a safety property too

Every tap is at a gauge terminal, every relay is in parallel, the original bezel is stored unmodified. If any of this turns out to be a bad idea in the car, it comes out in an afternoon and leaves the car stock. Spec §13 asks for photographs of every factory connector before it is disconnected, a wiring map of every tap, and bagged originals. That is not sentiment about the car; it is the escape hatch for the whole project.

What is *not* claimed

Honesty about limits is part of the posture:

- Nothing has been flashed to a board or connected to a car yet. Timing under UART floods, behaviour during flash writes, I2C fault recovery, DRM timeout recovery, watchdog reboot and
  real touch behaviour are all spec §11 rig tests still to run.
- The complete render stack is not certified allocation-free; NanoVG and the GPU driver have not been profiled on the target.
- Relay telemetry reports commanded outputs, not measured lamp output, door position, clutch
  engagement or blower speed.
- The command protocol has no transaction IDs; after a timeout the host stops sending until restart rather than risk misattributing a late ack. That is a known conservative choice, not a finished design.
- Sudden power loss can lose queued CSV rows or an unpersisted settings change.

How to think when adding something

Before adding a feature that touches the car, answer in writing:

1. Which processor decides, and why is that the right one?
2. If the wire between them is cut at that moment, what does each side do?

3. If this component fails open, fails closed, or freezes, what does the driver see and lose?
4. Does the factory control still work exactly as before?
5. Can it come out without a trace?

If an answer is uncomfortable, the design is not done. The spec's own phrase: "a hobby project on a valuable car. Correctness and reversibility beat cleverness and feature count every time."

---

10. Glossary

One line each. Grouped loosely; alphabetical within a group.

Electrical

- **ACC** — the ignition switch's accessory position/feed: live in ACC and RUN, dead in START and OFF. Triggers the master relay.
- **Active-high / active-low** — whether 3.3 V or 0 V means "on". Relay drives are active-high; the reverse input is active-low.
- **ADC** — analog-to-digital converter; turns a voltage into an integer count.
- **Ammeter** — a gauge in series with the charging circuit measuring current. Corvettes before 1976 have one. Never tap it.
- **ANL fuse** — a bolt-down high-current fuse used near the battery.
- **AWG** — American Wire Gauge; smaller number is thicker wire. 8 AWG main feed, 12 AWG Pi ground, 16 AWG MCU ground.
- **BAT54S** — a dual Schottky diode in one SOT-23 package, used to clamp each ADC input to the rails.
- **Buck converter** — a switching regulator that steps voltage down efficiently. Automotive-rated here.
- **Bulkhead connector** — the multi-pin connector where the engine harness passes through the firewall; cavities are numbered on the factory diagram.
- **Differential input** — an ADC channel that measures the voltage between two pins rather than one pin and ground. Cancels ground offset.
- **Flyback diode** — a diode across a relay coil that absorbs the voltage spike when the coil is switched off.
- **Ground offset** — the voltage difference between two points that are both nominally "ground". 100–200 mV between block and console on this car.
- **IGN** — the ignition feed: live in RUN and START. Feeds the gauges; sensed for Pi shutdown.

- **Load dump** — the voltage spike an alternator produces when its load is suddenly disconnected. Automotive parts are rated for it.
- **NTC thermistor** — a resistor whose resistance falls as temperature rises. The coolant sender.
- **Optocoupler** — an LED and a light-sensitive transistor in one package; passes a signal with no electrical connection.
- **PGA** — programmable gain amplifier; the ADS1115 setting that chooses what voltage equals full scale.
- **Piggyback tap** — connecting a high-impedance wire to an existing node (gauge–sender junction) to read it without altering the circuit.
- **Pull-up** — a resistor tying an input weakly to 3.3 V so it reads high until something pulls it low.
- **Relay** — an electromagnet (coil, pins 85/86) closing a separate contact (30 common, 87 normally open, 87a normally closed).
- **Rheostat** — a variable resistor used as a two-terminal device. The fuel float sender.
- **Schottky diode** — a diode that conducts at ~0.3 V instead of ~0.7 V; used for clamps because it acts first.
- **Sender** — the automotive term for a sensor that feeds a gauge. Fuel, oil pressure and coolant senders here.
- **SPS** — samples per second, the ADC conversion rate. 128 SPS here.
- **Star ground** — one ground point with a separate wire from each board, so no board's return current flows through another's wire.
- **Supercapacitor** — a capacitor large enough to power the Pi for a few seconds; the graceful-shutdown reserve.
- **TVS** — transient voltage suppressor; a diode that clamps spikes on the 12 V rail.
- **Voltage divider** — two resistors in series; the junction carries a fixed fraction of the input. 47k/10k for the volt channel.

Car

- **Birdcage** — the steel structure around the C3 cabin under the fibreglass body; the star ground point.
- **Blend door** — the door that mixes heated and unheated air; cable-operated on this car and kept.
- **Blower resistor pack** — series resistors in the heater box selected by the fan switch to set blower speed.
- **C3** — the third-generation Corvette, 1968–1982. This car is a 1975 coupe.
- **C60** — the GM option code for factory air conditioning.
- **Dimmer** — the floor switch selecting high or low beam.

- **Headlight doors** — the vacuum-operated covers over the pop-up headlamps; the vacuum valve is inside the headlight switch.
- **L48 / L82** — the base and high-performance 350 small-block engines; which one is installed is unconfirmed.
- **Mode doors** — the vacuum-actuated doors selecting vent, floor or defrost.
- **Points ignition** — the pre-electronic ignition system (1974 was the last year for points on Corvettes; the 1975 has HEI, but the spec's noise budgeting assumes the worse case). Either way, ignition noise is on every wire.
- **Vacuum switching valve** — the lever-operated valve routing engine vacuum to the mode-door actuators.

Microcontroller

- **Arduino** — here, the C++ framework and API (`digitalWrite` , `Wire` , `Serial1`), not a board.
- **DevKitC-1** — Espressif's ESP32-S3 development board. N8R8 means 8 MB flash, 8 MB PSRAM.
- **ESP-IDF** — Espressif's SDK, with FreeRTOS underneath the Arduino layer.
- **ESP32-S3** — the dual-core 240 MHz microcontroller running the firmware.
- **FreeRTOS** — the small real-time OS the ESP32 runs; used for the flash-write task.
- **GPIO** — general-purpose input/output pin.
- **Hysteresis** — setting a fault at one level and clearing it at another so it does not chatter.
- **I2C** — two-wire bus (SDA, SCL) with 7-bit addresses; both ADS1115s live on it.
- **IIR filter** — infinite impulse response; here a first-order low-pass `y += α(x − y)` with a per-channel time constant.
- **Median of 5** — take five samples, use the middle value; rejects single-sample spikes.
- **NVS** — non-volatile storage, the ESP32's key-value store in flash; holds saved calibration curves.
- **PlatformIO** — the command-line build system for the firmware; `pio run` , `pio run -t upload` .
- **pioarduino** — the community PlatformIO platform with a current toolchain (GCC 13, C++20).
- **Strapping pins** — GPIOs sampled at reset to select boot mode; avoided in the pin map.
- **τ (tau)** — a filter's time constant; after one τ the output has moved 63 % toward a step. Fuel 10 s, coolant 2 s, oil 0.2 s.
- **UART** — asynchronous serial: TX, RX, ground, agreed baud rate. 115200 8N1 here.

Protocol

- **8N1** — 8 data bits, no parity, 1 stop bit.
- **Ack** — the MCU's `{"ack":...,"ok":...}` reply to every command.
- **Baud** — bits per second on the wire.
- **CRC-16/CCITT-FALSE** — the checksum variant used: poly 0x1021, init 0xFFFF, check value 0x29B1.
- **Frame** — one telemetry line from the MCU, and the `Frame` struct it parses into.
- **Plausibility limit** — a range far outside operation used to reject corrupt frames; not a warning threshold.
- **Stall timeout** — the 1 s (configurable, ≤ 1000 ms) after which telemetry is stale.

Display host

- **ALSA** — the Linux kernel audio API.
- **bcm2835_wdt** — the Pi's hardware watchdog driver, `/dev/watchdog`.
- **CM4** — Raspberry Pi Compute Module 4: BCM2711, eMMC, no connectors, needs a carrier board.
- **CRTC** — the display controller's scan-out engine that a framebuffer is attached to.
- **DMA-BUF** — a kernel handle to a buffer several devices can share without copying.
- **DRM / KMS** — Direct Rendering Manager / Kernel Mode Setting; the kernel's GPU and display API.
- **EGL** — the glue between OpenGL ES and a native surface (GBM here).
- **EGLImage** — an EGL object wrapping an external buffer (a DMA-BUF) as a GL texture.
- **evdev** — the kernel input event interface, `/dev/input/eventN`.
- **GBM** — Generic Buffer Management; allocates scan-out buffers for EGL.
- **GLES2** — OpenGL ES 2.0, the GPU API NanoVG uses on the target.
- **HID** — USB Human Interface Device class; the touch digitiser is one.
- **libinput / libudev** — user-space libraries for input events and device discovery.
- **Mesa / v3d / vc4** — the open-source GPU driver stack; v3d is the 3D driver, vc4 the display driver.
- **NanoVG** — a small antialiased vector drawing library on top of GL; the only rendering abstraction.
- **Overlayfs** — a filesystem layer that makes a read-only root appear writable, with changes held in RAM.
- **Page flip** — swapping the framebuffer being scanned out, at vsync.
- **Plane** — a DRM layer the display controller composites during scan-out; multi-plane is the deferred stage 2.
- **poll(2)** — the POSIX call that waits on file descriptors with a timeout; the event loop's wait.
- **SDL2** — the desktop windowing library; never linked into the target build.

- **SPDIF** — a digital audio interconnect; optical here, to the amplifier.
- **SPSC queue** — single-producer single-consumer bounded queue.
- **Triple buffer** — three-slot wait-free snapshot exchange with an atomic index.
- **UVC** — USB Video Class; the rear camera.
- **V4L2 M2M** — Video4Linux2 memory-to-memory; the kernel interface to the hardware H.264 decoder.
- **Watchdog** — a hardware timer that reboots the board if not kicked periodically.

Product

- **Carlinkit CPC200-CCPA** — the wireless CarPlay USB dongle the design targets.
- **Dock** — the HVAC and lamp control area at the bottom of the screen.
- **HiFiBerry Digi+ Pro** — the Pi HAT providing SPDIF output.
- **Optical bonding** — laminating the touch glass to the LCD with no air gap; required for sunlight readability.
- **Sheet** — a panel (audio, settings, calibration) shown in the well.
- **Tier** — a rung of the simulation ladder (spec §9), 0 to 7.
- **Well** — the central rectangle showing CarPlay, the camera, or a sheet.

---

11. Talking Points

How to explain the project out loud, and the questions another engineer will ask.

The thirty-second version

"I'm putting a portrait touchscreen into a 1975 Corvette: CarPlay in the middle, oil, coolant, fuel and voltage gauges around it, and the heater and headlamp controls on screen. Two boards:
an ESP32 that taps the existing gauge senders, filters the readings, decides when something is dangerous and sounds a buzzer, and a Raspberry Pi CM4 that just draws the screen and hosts CarPlay. The ESP32 works with the Pi off. Nothing is cut: every sender is read at the gauge terminal and every relay is in parallel with a factory switch, so the whole thing comes out in an afternoon."

The whiteboard sequence

Draw it in this order; each step motivates the next.

1. **The car's gauge circuit.** 12 V → gauge → wire → sender → block. "The sender is a variable resistor; the gauge and sender form a divider." Mark the junction.

2. **The tap.** One wire from the junction to an ADC input. "High impedance, so the gauge never notices." Then: "But the block and my board disagree about ground by 200 mV, so I measure differentially against the sender's own ground. Four differential channels, two ADS1115s."
3. **The MCU.** Median of 5, IIR with a per-channel time constant, table lookup, hysteresis. Draw the LED and buzzer on GPIOs. "This path never touches the serial link."
4. **The wire.** Newline-delimited JSON with a CRC at 10 Hz. "Readable with `cat`, self-syncs after noise, extensible."
5. **The Pi.** NanoVG straight onto KMS, no X. Event loop, triple buffer, watchdog, read-only root. "It's an appliance."
6. **The relays.** Blower taps, clutch, two vacuum solenoids, headlamp relay across the switch. Do the headlamp failure table from memory.
7. **The ladder.** Browser mock → desktop C++ with a physics mock → breadboard → sender rig →
   replay → Pi on the bench → in-car on a bracket → permanent. "Tiers 0–5 need no car."

Questions you will get, with answers

Why not one Raspberry Pi with GPIO for everything?
Boot time (the oil warning must be live at key-on, Linux takes seconds), determinism (a general-purpose OS cannot bound the latency of a warning or a relay), electrical exposure (the board that touches the engine should be the cheap one), and blast radius (the Pi runs a reverse-engineered USB protocol and a graphics stack; it will crash, and the blower and buzzer must not care).

Why not just replace the gauges?
Reversibility. The factory gauges stay connected and working; we listen to the same node. If the project is abandoned or the car is sold, it goes back to stock. Also inspection: a working factory headlamp switch is a legal requirement in most places.

Why JSON over UART instead of a binary protocol or CAN?
At 10 Hz the bytes are irrelevant and readability is worth a lot on a fifty-year-old harness. Newlines make it self-synchronising after noise; a CRC-16 catches corruption; unknown keys are
skipped so it extends without versioning. CAN is right for a modern car with many modules; here there are two boards and no CAN anywhere on the vehicle, so it would add transceivers and
a stack for nothing. UART is a pin on the Pi's SoC, alive before USB is.

Why an external 16-bit ADC when the ESP32 has one?
The ESP32's ADC is 12-bit, noisy, nonlinear at the ends, and not usefully differential. The

ADS1115 has a real PGA and differential inputs for $4. Two chips because four differential channels need eight pins.

Why differential? Isn't ground ground?
Not on a fibreglass car. Every ground is a wire, wires have resistance, and 100–200 mV of offset between the block and the console is a 10–20 % error on a sender whose whole range is 0–90 Ω. Measuring each channel against its own sender's ground cancels it.

Why are the filter time constants different per channel?
Because the physics is different. Fuel sloshes, so 10 s. Coolant changes slowly, so 2 s. Oil pressure is the gauge that has to show a drop *now*, so 200 ms. Unifying them would filter the one failure that destroys an engine into invisibility.

Why does the buzzer depend on system voltage?
Key on, engine off: zero oil pressure, 12.6 V, normal, should be silent. Engine running: the alternator holds 13.8 V+, so above 13.2 V zero oil pressure is an emergency. The LED lights either way; only the buzzer is gated. Overheat sounds regardless.

Why hold HVAC state on comms loss instead of failing safe?
"Safe" for the electronics is "dangerous" for the driver: losing the blower in July or defrost in rain because a cable came loose. The MCU changes outputs only on command. Defrost also has
a mechanical or hard-wired path because a MCU reset initialises outputs off.

How do you know the gauge isn't showing an old value?
The Pi stamps every accepted frame with its own receive time and greys everything out after 1 s without one. Invalid frames don't count. A `LinkLoss` mock profile exercises this on every desk. Same for the camera: "unavailable", never a frozen frame.

Why CM4 and not Pi 5?
Hardware H.264 decode. CarPlay video is H.264, the CM4 decodes it in a dedicated block and hands the GPU a zero-copy buffer; the Pi 5 dropped that block and decodes in software, which is
more heat behind a dash. eMMC over SD is a bonus.

Why no X or Wayland? Why not Qt or Flutter?
One full-screen process owns the display; a compositor adds latency, packages and failure modes for nothing. NanoVG onto KMS is what kiosks and head units do. Frameworks bring a widget model that makes "draw stale as stale" harder, and a dependency surface that has to be justified on a read-only appliance. Immediate-mode drawing from a snapshot every frame is the simplest thing that honours the invariants.

Why C++ and not Rust, Python, Dart...?
C++20 is what runs natively on both the ESP32 toolchain and Bookworm's GCC 12 with the same
source files (the protocol parser and CRC compile unchanged on both). No runtime, no GC, bounded
allocation is expressible. Dart AOT was considered and rejected for the same reasons; a scripting language cannot make the no-heap, no-lock render-path promises.

Why no heap, no exceptions, no locks in the render loop?
`malloc` takes a lock and can page-fault; unwinding is unbounded; a mutex turns another thread's hiccup into a missed frame, and the watchdog is counting. Preallocate, return errors, and exchange state through a wait-free triple buffer.

Why three buffers and not two?
With two, the producer can wrap around onto the slot the consumer is still copying. Three keeps one slot per side plus one in the middle, and ownership moves by a single atomic exchange.

Why a read-only root filesystem?
Power loss mid-write is how Pis die. Read-only root with overlayfs removes the failure class. The two files that must persist live on a small `/data` partition written atomically.

How do you develop without the car or the hardware?
Three seams: `DisplayBackend`, `InputSource`, `SensorSource`. Desktop adapters (SDL2, mouse,
a physics mock ported from the browser mockup) and target adapters (KMS/GBM/EGL, libinput,
UART). Everything above the seams is identical. Recorded drives replay on the desk. Tiers 0–5 of the ladder need no car.

What if the relay sticks on?
Headlamps: lights stay on, battery drains, you are never dark. Blower: it runs. Clutch: the pressure switch is still in the loop. Nothing we added can remove a factory function; that is the design rule, not a hope.

What's the biggest risk?
Per spec §12: buying the wrong CarPlay dongle, a panel that is unreadable in sun (there is no software fix; optical bonding and 1000 nits or nothing), heat behind the dash, fuel calibration (tank geometry is nonlinear and needs measured fills), and scope. The gauges plus HVAC are a finishable project on their own; CarPlay is additive by design.

What is actually done?
The software through the desktop UI, mock physics, record/replay, settings persistence, the event loop and watchdog plumbing, the DRM and libinput backends (compiled, not run on a Pi),

and the firmware (compiled, not flashed). All of it passes tests on macOS and in the arm64 Bookworm container with zero clang-tidy findings. Nothing has touched a board or the car. Hardware acceptance (spec §11) is the remaining work, and it is gated on parts.

Honest phrases to use

- "The nominal GM curve is ... but every number gets calibrated on the bench."
- "Commanded state, not measured state" for anything the relay telemetry says.
- "The engine is a 350; whether it's the L48 or L82 is unconfirmed."
- "Design intent" for any timing claim about the MCU until it has been measured.
- "That's an open item" for command recovery after a timeout, the ignition shutdown handler, and the OS image.

Things not to say

- That the Pi "monitors the engine". It displays what the MCU reports.
- That the system "controls the headlights". It can turn them on in parallel; the switch is authoritative.
- That the gauges are "accurate". They will be calibrated; today they are placeholders and say so.
- That anything has been tested in the car.

> Copyright © 2026 Tyler R. Suehr